



Name: _____ Cohort/Batch: _____ Date: _____ Score: _____

SWAGGER/OPENAPI PRACTICE SHEET - 10 CONCEPTUAL Q&A

10 conceptual Q&A API Technologies

TOTAL: 10 QUESTIONS

Q1 What is Swagger/OpenAPI and what API communication problem does it solve? **Rest Model**

Q6 How do you implement pagination in a Swagger/OpenAPI API? **API API**

Q2 How does Swagger/OpenAPI define its schema or contract? **Structure**

Q7 How do you document a Swagger/OpenAPI API? **Lifecycle**

Q3 How does Swagger/OpenAPI handle authentication? **Configuration**

Q8 How do you test a Swagger/OpenAPI API? **Compliance**

Q4 How does Swagger/OpenAPI handle API versioning? **Hardening**

Q9 How do you protect a Swagger/OpenAPI API from abuse? **Testing**

Q5 How does Swagger/OpenAPI handle errors and status codes? **API**

Q10 What monitoring and observability should a Swagger/OpenAPI API have? **API**



ANSWER KEY & EXPLANATORY SOLUTIONS

Comprehensive verified answers, best-practice configs, security controls & reference

VERIFIED SOLUTIONS

A1

Swagger/OpenAPI defines a protocol or standard for

Mitigation

Swagger/OpenAPI defines a protocol or standard for structured communication between services. It addresses concerns like schema definition, payload format, versioning, or transport efficiency that plain HTTP with ad-hoc JSON does not solve on its own.

A6

Cursor-based pagination (next_cursor token) is preferred for

Security

Cursor-based pagination (next_cursor token) is preferred for large or frequently-updated datasets because it is stable. Offset-based pagination (page, limit) is simpler but can skip or duplicate rows if data changes between requests.

A2

Swagger/OpenAPI uses an IDL or schema language

Primitives

Swagger/OpenAPI uses an IDL or schema language to declare types, operations, and their inputs and outputs. This schema is the single source of truth from which documentation, validation, and client SDKs can be auto-generated.

A7

Swagger/OpenAPI APIs use an API specification (OpenAPI)

Validation

Swagger/OpenAPI APIs use an API specification (OpenAPI, AsyncAPI, GraphQL SDL) to generate interactive documentation (Swagger UI, ReDoc, GraphQL). Good docs include examples, error codes, and authentication instructions.

A3

Swagger/OpenAPI APIs commonly use Bearer tokens

Hardening

Swagger/OpenAPI APIs commonly use Bearer tokens (JWT/OAuth2) in the Authorization header or API keys in custom headers. Transport-level security (TLS) is always required. Additional mechanisms (mTLS, HMAC) apply to high-trust scenarios.

A8

Contract testing (Pact) verifies the provider matches

Governance

Contract testing (Pact) verifies the provider matches the consumer's schema. Integration tests call real endpoints against a test environment. Load tests (k6, Gatling) verify performance under production-like concurrency.

A4

Common versioning strategies include URL path (/v1/users)

Gas

Common versioning strategies include URL path (/v1/users), request headers (Accept: application/vnd.api+v2+json), and query params (?version=2). Swagger/OpenAPI prefers one strategy and maintains backward compatibility within a major version.

A9

Rate limiting (tokens per IP per minute)

Assessment

Rate limiting (tokens per IP per minute) prevents DoS. Input validation rejects malformed requests before they reach business logic. Output filtering (response schema) prevents accidental data leakage. Monitor for anomalous usage patterns.

A5

Swagger/OpenAPI uses HTTP status codes (200 OK,

Federation

Swagger/OpenAPI uses HTTP status codes (200 OK, 400 Bad Request, 401 Unauthorized, 404 Not Found, 500 Internal Server Error) combined with a structured error body containing a code, message, and optional details field.

A10

Instrument request count, latency histograms, and error

Optimization

Instrument request count, latency histograms, and error rate with Prometheus or a cloud metrics system. Add distributed tracing with OpenTelemetry. Log structured request/response data. Set SLOs and alert when SLIs breach them.